

8.3.2 Eigene Objekte erzeugen

Bisher haben wir nur mit den Objekten gearbeitet, die im Level vorhanden sind. Das werden wir nun ändern.



Aufgabe 1

- a) Öffnen Sie **Level 32** und versuchen Sie es **mit den Pfeiltasten** zu lösen.
- b) Möglicherweise kennen Sie bereits einige Tricks, um das Level dennoch zu lösen. Probieren Sie (maximal eine Minute) einige davon aus, machen Sie aber ggf. **Änderungen am Framework wieder rückgängig!** Ansonsten überspringen Sie diesen Teil einfach.
- c) Sie können das Level lösen, indem Sie dem Bogenschützen Hindernisse in den Weg stellen. Dazu erzeugen Sie ein neues Hindernis-Objekt und stellen es an die gewünschte Stelle.

Testen Sie:

```
tor1 = Tor(1,2,True)
level.objekt_hinzufuegen(tor1)
```

- d) **Lösen** Sie jetzt **Level 32**, indem Sie vor die Bogenschütze Objekte setzen.

Aufgabe 2

- a) Testen Sie den folgenden, verkürzten Befehl:

```
level.objekt_hinzufuegen(Tuer(1,2))
```
- b) Welcher Unterschied besteht zwischen der langen und der kurzen Fassung?
 Angenommen, Sie würden jetzt gerne die Farbe der Tür abfragen: Welche Variante würden Sie dann bevorzugen?
- c) Öffnen Sie **Level 33** und versuchen Sie mithilfe des angepassten Klassendiagramms einige Monster hinter den Bogenschützen erscheinen zu lassen. Mal sehen, was dann wohl passiert.

Monster

```
x : Integer
y : Integer
richtung : String
ist_ko : Boolean
Monster(x,y : int, r : str)
ist_passierbar() : Boolean
setze_richtung(r : String)
...
```

Merke:

Objekte werden durch den Aufruf des **Konstruktors** einer Klasse erzeugt. Dieser heißt immer wie die Klasse selbst und verlangt in der Regel verschiedene Parameter, welche im Klassendiagramm ablesbar sind.

Allgemein

Klassenname
attribut_1 : Datentyp
attribut_2 : Datentyp
...
attribut_n : Datentyp
Konstruktor (Parameter)
methode_1(...) : ...
methode_2(...) : ...
...

Beispiel

Monster
x : Integer
y : Integer
richtung : String
ist_ko : Boolean
Monster (x,y : int, r : str)
ist_passierbar() : Boolean
setze_richtung(r : String)
...

Der **Aufruf** des Konstruktors **erzeugt** dann das Objekt, so dass es z.B. an eine **Variable** gebunden werden kann: **obj** = **Konstruktor**(...). Im Beispiel: **m** = **Monster**(1,2,"up").

Aufgabe 3

a) Öffnen Sie **Level 34**. Leider fehlt der Schlüssel, um das Level zu lösen.

b) Betrachten Sie das Klassendiagramm der Klasse Schlüssel:

Schlüssel
x : int
y : int
farbe : str
Schlüssel (x,y : int)
get_farbe() : String
set_farbe(f : String)
setze_position(x,y)

c) Lösen Sie mit diesem Wissen **Level 34**, indem Sie selbst einen passenden Schlüssel erzeugen.

Aufgabe 4

„Bauen“ Sie das folgende Level nach. Nutzen Sie Level 1 als Grundlage sowie die folgenden Signaturen:

- **Knappe**(x,y : int, r : str)
- **Tuer**(x,y : int, farbe : str)
- **Schluessel**(x,y : int)
- **Herz**(x,y : int)
- **Hindernis**(art : str, x,y : int)

Es gibt Bäume, Berge und Büsche im Level.

